Package ‘soundmeteR’

November 5, 2025

Type Package

Title R package for calculating sound pressure and intensity levels

Version 1.0.0

Author
Cássio Rachid Simões, Ingrid Maria Denóbile Torres, Gabriel L. M. Rosa, Wilmara Guedes de Lucena, Shaka Nóbrega Marinho Furtado, Marcelo Felgueiras Napoli and Carlos Barros de Araújo

Maintainer Cássio Rachid Simões <cassiorachid@gmail.com>

Description
An R package for sound analysis using octave and one-third octave spectral analysis, with calibration support for obtaining Sound Pressure Level (SPL) and Sound Intensity Level (SIL).

URL https://github.com/ConservaSom/soundmeteR

BugReports https://github.com/ConservaSom/soundmeteR/issues

License GPL (>=3)

Depends R (>= 3.5.0), tuneR, seewave, dplyr, progress

Encoding UTF-8

LazyData TRUE

RoxygenNote 7.3.3

Contents

---

calibration *Extract calbration factor from reference sinal*

Description

Function to compute the calibration factor from a reference signal, converting measured levels from decibels at full scale (dBFS) to absolute values (dB) for subsequent analyses.

Usage

```
calibration(
  file = NULL,
  channel = "left",
  from = 0,
  to = Inf,
  leqsignal = NA,
  weighting = "none",
  ref = 20
)
```

Arguments

`file`	Specifies audio file to be analyzed. It can be a single file name on the computer or a Wave object on the environment (by default NULL). Only ".wav" files are accepted.
`channel`	Character. Choose “left” or “right” channel (by default "left"). Argument passed to mono function from tuneR to extract the desired channel.
`from`	Numeric. The start time (in seconds) of the segment to analyze. It can also be relative to the end of the file (in negative values). See examples for details.
`to`	Numeric. The end time (in seconds) of the segment to analyze. Could also be relative to the beginning of the file (in negative values). See examples for details.
`weighting`	Character. Defines the weighting curve for the analysis, passed to the `dBweight` function. Accepted values are "A", "B", "C", "D", "ITU", and "none" (by default "none"). See `dBweight` for details. #' @param leqsignal Numeric. Specifies the sound pressure level (in dB SPL) for the signal in the audio (by default `NA`).
`ref`	Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air,it is 20 µPa (by default 20).

`dBtoLinear` *Convert deciBels scales to linear*

Description

Function to convert decibels (dB) to linear values (µPa).

Usage

```
dBtoLinear(x, factor="IL", ref=1)
```

Arguments

`x`	Numeric. A numeric vector or a numeric matrix with dB values.
`factor`	Character. Specify the factor to use for conversion. Use `SPL` (Sound Pressure Level) for amplitude-base data, with factor `20`, or `IL` (Intensity Level) for power-base data, with fator `10` (by default "IL").
`ref`	Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air,it is 20 µPa (by default 1).

Details

For further details on selecting the appropriate factor, we recommend consulting this web page.

Value

The same input object with the values converted.

See Also

`rms.dB`, `LineartodB`, `convSPL`

Examples

```
dBtoLinear(c(80,60,65,62))
LineartodB(dBtoLinear(c(80,60,65,62)))
```

freq.bands *Interval Limits*

Description

Function to compute frequency intervals based on patterns defined by the user. This function is adapted from the `octaves` functions from `seewave` package.

Usage

```
freq.bands(x, interval = 2, below = 3, above = 3)
```

Arguments

`x`	Numerical. Frequency used as the central point from which bands are calculated in kHz (by default 2).
`interval`	Numeric. Specifies the interval pattern to be applied. See details for further information.
`below`	Numerical. Number of intervals below x (by default 3).
`above`	Numerical. Number of intervals above x (by default 3).

Details

The specified interval is applied as x/interval (for the lower bound) and x×interval (for the upper bound). Some examples of intervals that can be applied are:

- Octaves = 2
- Third of an octave = 2^(1/3)
- Perfect fifth (music theory) = 3/2
- Major third (music theory) = 5/4
- Minor third (music theory) = 6/5
- For additional values that can be used, please refer to this link.

Value

A numeric vector containing the frequency limits of each interval.

See Also

`octaves`

Examples

```
freq.bands(1000, interval=2, below = 1, above = 1) #octaves
freq.bands(1000, interval=2^(1/3), below = 3, above = 3) #Third of octaves

#https://academics.hamilton.edu/music/spellman/class_notes/music_theory.htm
freq.bands(440, interval=3/2, below = 0, above = 1) #Perfect fifth (music theory) of A 440
freq.bands(440, interval=5/4, below = 0, above = 1) #Major third (music theory) of A 440
freq.bands(440, interval=6/5, below = 0, above = 1) #Minor third (music theory) of A 440
```

---

leqbands — *Equivalent Level per octaves or one-third octaves*

Description

Function to comput the Equivalent Level (Leq) across octaves or one-third octaves.

Usage

```
leqbands(
  files = "wd",
  channel = "left",
  from = 0,
  to = Inf,
  weighting = "none",
  bands = "thirds",
  ref = 20,
  Leq.calib = NULL,
  Calib.value = NULL,
  saveresults = F,
  outname = NULL,
  progressbar = T
)
```

Arguments

`files`	Specifies audio file(s) to be analyzed. It can be set to "wd" to select all ".wav" files in the work directory, a single file name, a character vector with multiple file names, a Wave object, or a list of Wave objects (by default "wd"). Only ".wav" files are accepted.
`channel`	Character. Choose “left” or “right” channel (by default "left"). Argument passed to mono function from tuneR to extract the desired channel.
`from`	Numeric. The start time (in seconds) of the segment to analyze. It can also be relative to the end of the file (in negative values). See examples for details.
`to`	Numeric. The end time (in seconds) of the segment to analyze. Could also be relative to the beginning of the file (in negative values). See examples for details.

weighting	Character. Defines the weighting curve for the analysis, passed to the dBweight function. Accepted values are "A", "B", "C", "D", "ITU", and "none" (by default "none"). See dBweight for details.
bands	Character. Use "octaves" for octave bands or "thirds" for one-third octave bands intervals (by default "thirds").
ref	Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air,it is 20 µPa (by default 20).
Leq.calib	Numeric. Specifies the sound pressure level (in dB SPL) for the signal in the audio (by default NULL). Cannot be set if Calib.value is specified.
Calib.value	Numeric. Specifies the calibration value (by default NULL). Can not be set if Leq.calib is specified.
saveresults	Logical. Set to TRUE to save the results to a .txt file (by default FALSE).
outname	Character. If saveresults is TRUE, specifies a name to append to the file name in the txt file (by default NULL).
progressbar	Logical. Set TRUE to display a progress bar showing elapsed time and the last completed file number, or FALSE to hide it (default TRUE).

Details

Caution: Ensure that the audio file has a duration with whole second values to avoid bugs. For example: 35s, 60s, 19s. By default, the function will truncate your audio file to the next whole second.

This function only works with mono audio files, and the audio must have a sampling rate of at least 44,100 Hz.

If you are working with decibels at full scale (dBFS), we recommend setting ref=1 so the results will be relative to 0 dBFS.

References

Power spectrum adapted from: Carcagno, S. 2013. Basic Sound Processing with R [Blog post]. Retrieved from http://samcarcagno.altervista.org/blog/basic-sound-processing-r/

Miyara, F. 2017. Software-Based Acoustical Measurements. Springer. 429 pp. DOI: 10.1007/978-3-319-55871-4

Examples

```
data(tham)
leqbands(tham)
leqbands(tham, Calib.value=130.24)
leqbands(tham, Calib.value=130.24, weighting="A")
leqbands(tham, Calib.value=130.24, weighting="A", bands="octaves")

leqbands(tham, Leq.calib=48.17, weighting="A")

leqbands(tham, from=-3.49, to=-0.9, ref=1) #dB at Full Scale
leqbands(tham, from=-3.49, to=-0.9, Calib.value=130.24) #song Sound Pressure Level
leqbands(tham, from=0, to=3.8, Calib.value=130.24) #background Sound Pressure Level
```

`leqbands_cal` *leqbands analysis for audiofiles with reference signal*

Description

This function passes the parameters to `leqbands` to automate the calibration process and return spectral analysis results in dB SPL.

Usage

```
leqbands_cal(
  files = "wd",
  channel = "left",
  from = 0,
  to = Inf,
  CalibPosition = NULL,
  CalibValue = NULL,
  ref = 20,
  weighting = "none",
  bands = "thirds",
  saveresults = F,
  outname = NULL,
  progressbar = T
)
```

Arguments

`files`	Specifies audio file(s) to be analyzed. It can be set to "wd" to select all ".wav" files in the work directory, a single file name, a character vector with multiple file names, a Wave object, or a list of Wave objects (by default "wd"). Only ".wav" files are accepted.
`channel`	Character. Choose “left” or “right” channel. Argument passed to mono function from tuneR (by default "left").
`from`	Numeric. The start time (in seconds) of the segment to analyze. It can also be relative to the end of the file (in negative values). See examples for details.
`to`	Numeric. The end time (in seconds) of the segment to analyze. It can also be relative to the beginning of the file (in negative values). See examples for details.
`CalibPosition`	Numeric. Specifies the calibration position. It can be a negative (relative to the sound file duration) or a positive value, or a data.frame containing these combinations. This parameter is used in conjunction with `CalibValue`.
`CalibValue`	Numeric. Specifies the calibration value. If `CalibPosition` is provided, it serves as the reference value. If `CalibPosition` is absent, `CalibValue` is used as the calibration reference.

`ref`	Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air,it is 20 µPa (by default 20).
`weighting`	Character. Defines the weighting curve for the analysis, passed to the dBweight function. Accepted values are "A", "B", "C", "D", "ITU", and "none" (by default "none"). See dBweight for details.
`bands`	Character. Use "octaves" for octave bands or "thirds" for one-third octave bands intervals (by default "thirds").
`saveresults`	Logical. Set to `TRUE` to save the results to a .txt file (by default `FALSE`).
`outname`	Character. If `saveresults` is `TRUE`, specifies a name to append to the file name in the txt file (by default `NULL`).
`progressbar`	Logical. Set `TRUE` to display a progress bar showing elapsed time and the last completed file number, or `FALSE` to hide it (default `TRUE`).

See Also

`leqbands`

`LineartodB` *Convert linear scales to deciBels*

Description

Function to convert linear values (µPa) to decibels (dB).

Usage

```
LineartodB(x, factor="IL", ref=1)
```

Arguments

`x`	Numeric. A vector or matrix containing linear values (in µPa).
`factor`	Character. Specify the factor for conversion. Use `SPL` (Sound Pressure Level) for amplitude-base data, with factor `20`, or `IL` (Intensity Level) for power-base data, with factor `10` (by default `IL`.)
`ref`	Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air,it is 20 µPa (by default 1).

Details

For details about the factor choice, we recommend consulting this web page.

Value

The same input object with the values converted.

See Also

rms.dB, dBtoLinear, convSPL

Examples

```
dBtoLinear(c(80,60,65,62))
LineartodB(dBtoLinear(c(80,60,65,62)))
```

---

pwrspec *Power Spectrum of a sound file*

Description

This functions computes a power spectrum from a sound file.

Usage

```
pwrspec(
  file,
  channel = "left",
  from = 0,
  to = Inf,
  bandpass = c(0, Inf),
  res.scale = "microPa",
  ref = 1
)
```

Arguments

`file`	A wave file path or a class Wave object already loaded into R environment.
`channel`	Character. Choose “left” or “right” channel. Argument passed to mono function from tuneR (by default "left").
`from`	Numeric. The start time (in seconds) of the segment to analyze. It can also be relative to the end of the file (in negative values). See examples for details.
`to`	Numeric. The end time (in seconds) of the segment to analyze. It can also be relative to the beginning of the file (in negative values). See examples for details.
`bandpass`	A vector of length two specifying the lower and upper limits of the bandpass interval in Hz.
`res.scale`	Character. Specifies the scale for adjusting the power spectrum amplitude. Use `microPa` for linear values in µPa, or `dB` for decibel values in dB SPL (by default `microPa`).
`ref`	Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air,it is 20 µPa (by default 1).

Value

A data.frame with columns:

- Freq.Hz: numeric, frequency in Hertz
- Amp.microPa: numeric, amplitude in μPa (present when res.scale = "microPa")
- Amp.dB: numeric, amplitude in dB SPL (present when res.scale = "dB")

References

Power spectrum adapted from: Carcagno, S. 2013. Basic Sound Processing with R [Blog post]. Retrieved from http://samcarcagno.altervista.org/blog/basic-sound-processing-r/

Miyara, F. 2017. Software-Based Acoustical Measurements. Springer. 429 pp. DOI: 10.1007/978-3-319-55871-4

Examples

```
data(tham)
pwrspec(tham)

#with from, to and bandpass
pwrspec(tham, from=3.8, to=7.1, bandpass=c(900,2000))

#from and to relative to duration (negatives values from the end of the soundfiles)
pwrspec(tham, from=-3.49, to=-0.9, bandpass=c(900,2000))
```

`rms.dB` *Root Mean Square with dB values*

Description

Function to compute the root mean square (RMS) of values in decibels (dB).

Usage

```
rms.dB(x, level = "SPL", na.rm = FALSE)
```

Arguments

`x`	Numeric. A vector or a matrix with decibels values (dB).
`level`	Character. Specify the factor for conversion. Use `SPL` (Sound Pressure Level) for amplitude-base data, with factor 20, or `IL` (Intensity Level) for power-base data, with fator 10 (By default `SPL`).
`na.rm`	Logical. Argument passed to `mean`. If `TRUE` removes NA (By default `FALSE`).

Details

This function converts the decibels data to linear values using the dBtoLinear function, computes the Root Mean Square (rms), and then converts the result back to decibels using the LineartodB function.

This function was adapted from the meandB and rms functions from seewave package. See their documentation for more details.

Value

A numeric value that representing the root mean square of x.

See Also

meandB, rms

Examples

```
rms.dB(c(80,60,65,62))
```

---

song.level *RMS from a sample of a sound file*

Description

RMS (Root Mean Square) from a sample of a sound file.

Usage

```
song.level(
  files = "wd",
  channel = "left",
  from = 0,
  to = Inf,
  freq.interval = c(0, Inf),
  fdom.int = c(0, Inf),
  wl = 512,
  ovlp = 50,
  CalibPosition = NULL,
  CalibValue = NULL,
  freq.weight = "none",
  ref = 20,
  progressbar = T
)
```

Arguments

- `files` — Specifies audio file(s) to be analyzed. It can be set to "wd" to select all ".wav" files in the work directory, a single file name, a character vector with multiple file names, a Wave object, or a list of Wave objects (by default "wd"). Only ".wav" files are accepted.
- `channel` — Character. Choose “left” or “right” channel. Argument passed to mono function from tuneR (by default "left").
- `from` — Numeric. Specifies the start time (in seconds) of the segment to analyze. It can also be relative to the end of the file (in negative values). See examples for details.
- `to` — Numeric. Specifies the end time (in seconds) of the segment to analyze. It can also be relative to the beginning of the file (in negative values). See examples for more details.
- `freq.interval` — Specifies the frequency interval for computing the RMS. It can be a vector of length two, with the lower and upper frequency bounds (in Hz), or a pattern to calculate a interval (e.g., octaves). For more details, refer to freq.bands.
- `fdom.int` — A vector of length two specifying the lower and upper frequency bounds (in Hz) to determine the dominant frequency. This frequency will be used as the center of the interval only if a pattern is specified in `freq.interval`.
- `wl` — Numeric. Window length for the analysis. It must be a even number of points (by default 512).
- `ovlp` — Numeric. Overlap percentage between two successive windows (by default 50). Argument passed to meanspec function from seewave.
- `CalibPosition` — Numeric. Specifies the calibration position. It can be a negative (relative to the sound file duration) or a positive value, or a data.frame containing these combinations. This parameter is used in conjunction with `CalibValue`.
- `CalibValue` — Numeric. Canbe a value to apply of the ref value from a calib signal (specified by CalibPosition).
- `freq.weight` — Character. Defines the weighting curve for the analysis, passed to the dBweight function. Accepted values are "A", "B", "C", "D", "ITU", and "none" (by default "none"). See dBweight for details.
- `ref` — Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air,it is 20 µPa (by default 20).
- `progressbar` — Logical. Set to `TRUE` to display a progress bar showing elapsed time and the last completed file number, or `FALSE` to hide it (default `TRUE`).

Examples

```
song.level(tham, freq.interval=c(22, 20000))

song.level(tham, freq.interval=c(22, 20000), CalibValue = 130.24)

song.level(tham, freq.interval=c(22, 20000), CalibValue = 130.24, freq.weight="A")

song.level(tham, freq.interval=c(22, 20000), CalibValue = 130.24, freq.weight="B")
```

```
song.level(tham, freq.interval=c(22, 20000), CalibValue = 130.24, freq.weight="C")

song.level(tham, from = 3.883035, to=7.044417, freq.interval=c(22.09429, 22627.38), CalibValue = 130.24, freq.weig

#Perfect fifth (music theory)
song.level(tham, fdom.int = c(800, 2000), from = 3.8, to=7, freq.interval=3/2, CalibValue = 130.24, freq.weight="A"

#Perfect fifth (music theory)
song.level(tham, fdom.int = c(800, 2000), from = 3.8, to=7, freq.interval=3/2, CalibValue = 130.24, freq.weight="A"
```

---

soundmeter *Sound meter measurements*

Description

Funtion that performs sound meter measurements.

Usage

```
soundmeter(
  files = "wd",
  channel = "left",
  from = 0,
  to = Inf,
  CalibPosition = NULL,
  CalibValue = NULL,
  ref = 20,
  fw = "none",
  bands = "octaves",
  tw = "fast",
  saveresults = F,
  outname = NULL,
  bandpass = c(0, Inf),
  progressbar = T
)
```

Arguments

`files`	Specifies audio file(s) to be analyzed. It can be set to "wd" to select all ".wav" files in the work directory, a single file name, a character vector with multiple file names, a Wave object, or a list of Wave objects (by default "wd"). Only ".wav" files are accepted.
`channel`	Character. Choose "left" or "right" channel. Argument passed to mono function from tuneR (by default "left").

`from`	Numeric. Specifies the start time (in seconds) of the segment to analyze. It can also be relative to the end of the file (in negative values). See examples for details.
`to`	Numeric. Specifies the end time (in seconds) of the segment to analyze. It can also be relative to the beginning of the file (in negative values). See examples for more details.
`CalibPosition`	Numeric. Specifies the calibration position. It can be a negative (relative to the sound file duration) or a positive value, or a data.frame containing these combinations. This parameter is used in conjunction with `CalibValue`.
`CalibValue`	Numeric. Specifies the calibration value (by default NULL). If "CalibPosition" is provided, it serves as the reference value. If "CalibPosition" is absent, "CalibValue" is used as the calibration reference.
`ref`	Numeric. The reference value for dB conversion. For sound in water, the common reference is 1 µPa, and for sound in air, it is 20 µPa (by default 20).
`fw`	Character. Defines the weighting curve for the analysis, passed to the dBweight function. Accepted values are "A", "B", "C", "D", "ITU", and "none" (by default "none"). See `dBweight` for details.
`bands`	Character. Use "octaves" for octave bands or "thirds" for one-third octave bands intervals (by default "thirds").
`tw`	Character. Specifies the time window to be used. Accepted values are "fast" and "slow" (default "fast").
`saveresults`	Logical. Set `TRUE` to save the results to a .txt file (by default `FALSE`).
`outname`	Character. If `saveresults` is `TRUE`, specifies a name to append to the file name in the txt file (by default `NULL`).
`bandpass`	Numeric. A vector of length two specifying the lower and upper limits of the bandpass interval in Hz.
`progressbar`	Logical. Set to `TRUE`) to display a progress bar showing elapsed time and the last completed file number, or `FALSE` to hide it (default is `TRUE`).

Details

If your reference signal is in a separate file, we recommend obtaining the `CalibValue` using the leqbands function. Examples in the documentation provide further details.

Examples

```
data("tham")
soundmeter(tham, CalibValue = 130.24, tw = "slow") # slow time window with calib value
soundmeter(tham, CalibValue = 130.24, tw = "fast") # fast

soundmeter(tham, CalibValue = 130.24, tw = "fast", fw = "A") # fast with frequency weight

soundmeter(tham, CalibValue = NULL, tw = "fast", ref = 1) # fast time window in dBFS
```

`sumdB` *Sum with dB values*

Description

Sum with dB values

Usage

```
sumdB(x, level = "IL", na.rm = FALSE)
```

Arguments

`x`	Numerical. A vector or a matrix with decibels values (dB).
`level`	Character. Specify the factor for conversion. Use `SPL` (Sound Pressure Level) for amplitude-base data, with factor 20, or `IL` (Intensity Level) for power-base data, with fator 10 (By default `IL`).
`na.rm`	Logical. Argument passed to `sum`. If `TRUE` removes NA (by default `FALSE`).

Details

Function to compute the sum of decibels values (dB).

This function converts the decibels data to linear values using the `dBtoLinear` function, computes the sum, and then converts the result back to decibels using the `LineartodB` function.

Value

A numeric value representing the sum of x.

See Also

`moredB`

Examples

```
sumdB(c(80,60,65,62))
sumdB(c(30,30), level="IL")
sumdB(c(30,30), level="SPL")
```

tham — Thamnophilus stictocephalus *song in a landscape*

Description

Thamnophilus stictocephalus song in a landscape

Usage

```
data(tham)
```

Format

An object of class "Wave"; see readWave.

Details

A landscape recording presenting *Thamnophilus stictocephalus* song between 3.883035 and 7.044417 seconds. The "calib.value" for this record is 130.24.

Source

Recording by Ingrid Maria Denóbile Torres

Tweighting — *Time Weighting of a Audiofile*

Description

This function split your audiofile into smaller segments, defined as `fast` (0.125s) and `slow` (1s), and analyze each one using the `leqbands` function.

Usage

```
Tweighting(
  file,
  window = "fast",
  channel = "left",
  weighting = "none",
  bands = "thirds",
  ref = 20,
  Calib.value = NULL,
  bandpass = c(0, Inf),
  from = 0,
  to = Inf
)
```

Arguments

`file`	Wave object
`window`	Character. Specifies the time window to be used. Accepted values are `fast` and `slow` (default `fast`).
`channel`	Character. Choose “left” or “right” channel. Argument passed to mono function from tuneR (by default "left").
`weighting`	Character. Specifies the end time (in seconds) of the segment to analyze. It can also be relative to the beginning of the file (in negative values). See examples for more details.
`bands`	Character. Use "octaves" for octave bands or "thirds" for one-third octave bands intervals (by default "thirds").
`ref`	Numeric. Defines the weighting curve for the analysis, passed to the dBweight function. Accepted values are "A", "B", "C", "D", "ITU", and "none" (by default "none"). See dBweight for details.
`Calib.value`	Numeric. Specifies the calibration value (by default NULL). Cannot be set if Leq.calib is specified.
`bandpass`	Numeric. A vector of length two specifying the lower and upper limits of the bandpass interval in Hz.
`from`	Numeric. Specifies the start time (in seconds) of the segment to analyze. It can also be relative to the end of the file (in negative values). See examples for details.
`to`	Numeric. Specifies the end time (in seconds) of the segment to analyze. It can also be relative to the beginning of the file (in negative values). See examples for more details.
`Leq.calib`	Numeric. Specifies the sound pressure level (in dB SPL) for the signal in the audio file (by default NULL). Cannot be set if Calib.value is specified.

See Also

`leqbands`, `soundmeter`

Examples

```
# creating an example sound file
som <- sine(1000, duration = 44500)

# default options without calibration (results in dBFS)
Tweighting(som)

# Simulation of a calib signal with a Leq of 94dB in the field ####
# fast
Tweighting(som, window = "fast", bands = "octaves", Leq.calib = 94)
# slow
Tweighting(som, window = "slow", bands = "octaves", Leq.calib = 94)

# Using the result of the simulation above to calibrate the sound and output
```

```
# fast
Tweighting(som, window = "fast", bands = "octaves", Calib.value = 309.67)
# slow
Tweighting(som, window = "slow", bands = "octaves", Calib.value = 309.67)

# With tham data
data(tham)
Tweighting(tham, window = "fast", bands = "octaves", Calib.value = 130.24) # fast
Tweighting(tham, window = "slow", bands = "octaves", Calib.value = 130.24) # slow
```

Index